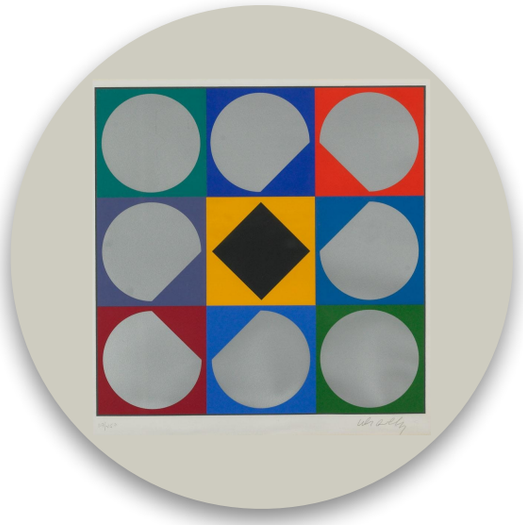
Generation Algorithms

Combinatorial Algorithms

Valentin Cassano

Guangdong Technion Israel Institute of Technology

Permutations

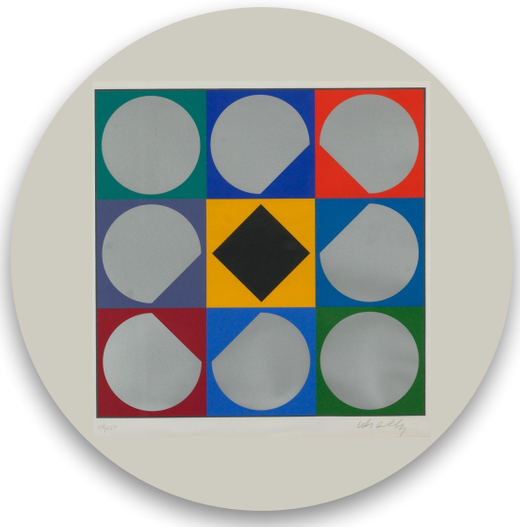


Victor Vasarely

We continue the chapter on the generation of combinatorial objects by looking at the generation of the $n!$ permutations of the set $[0 : n)$ in lexicographic order.

We begin by introducing some notation. A permutation of $[0 : n)$ is a sequence π of length n where every element in the set appears only once. We use Π^n to indicate the set of all permutations of $[0 : n)$ in lexicographic order.

Permutations



Victor Vasarely

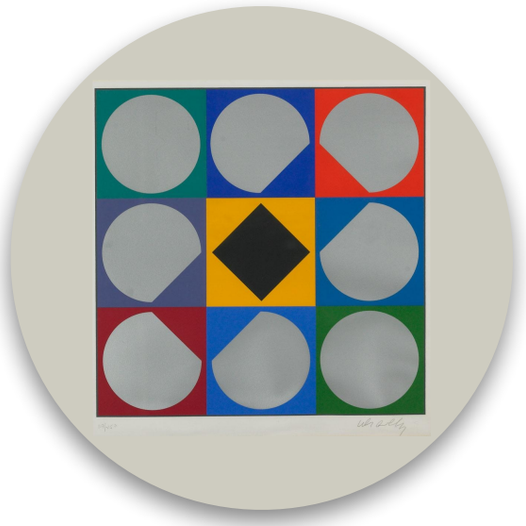
Let us take a closer look at Π^3. The set $[0:3) = \{0,1,2\}$ has $3! = 6$ permutations. Listed in lexicographic order they are:

i	π_i
0	[0, 1, 2]
1	[0, 2, 1]
2	[1, 0, 2]
3	[1, 2, 0]
4	[2, 0, 1]
5	[2, 1, 0]

The lexicographic order is clear: compare the first element; if equal, compare the second; if still equal, compare the third; and so on. So, the first permutations all start with 0; among those starting with 0, we have $[0,1,2]$ precedes $[0,2,1]$ because $1 < 2$ at the second position. Permutations starting with 1 follow all those starting with 0, and those starting with 2 come last.

Rank and unrank functions are then bijections from the first to second column, and viceversa: $\mathbf{rank}([1,0,2]) = 2$, $\mathbf{unrank}(4) = [2,0,1]$, and so on. The goal is to build efficient algorithms for these functions.

Permutations



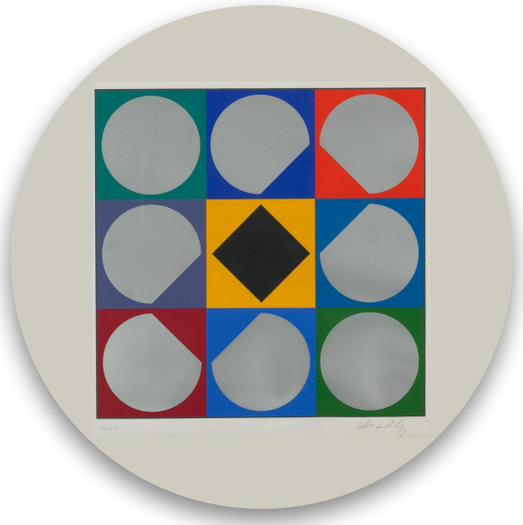
Victor Vasarely

In what follows, we will show that the place of a permutation in the lexicographic order is, in fact, a number expressed in factorial form.

By converting a number $r : [0 : n!)$ to its factorial form, we obtain a sequence of numbers that can be converted to a permutation in Π^n straightforwardly via Lehmer codes.

This view gives rise to a natural bijection between $[0 : n!)$ and Π^n.

PERMUTATIONS



Victor Vasarely

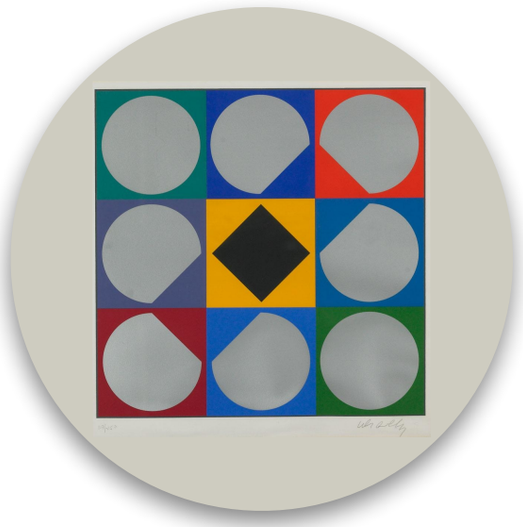
Let us begin with a revision of the decimal number system.

In the decimal system we have, for instance:

radix	10	10	10	10
digit range	[0:9]	[0:9]	[0:9]	[0:9]
digit position (msd to lsd)	0	1	0	3
digit position (lsd to msd)	3	2	1	0
multiplier	$100 = 10^3$	$100 = 10^2$	$10 = 10^1$	$1 = 10^0$

The radix is the upper bound of the range for possible digits. In the decimal system, all digits have the same radix. The number is the sum of the digits multiplied by the multipliers.

Permutations



Victor Vasarely

To achieve a direct one-to-one correspondence between permutations of and numbers, we turn to the factoradic, or factorial number system.

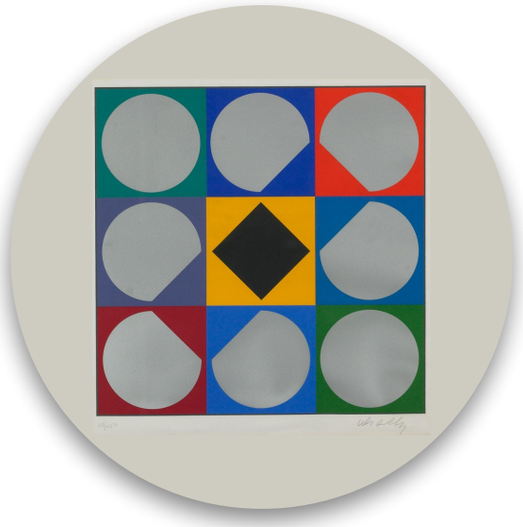
In this number system, the radix of a digit depends on the factorial of the position. For instance:

radix	4	3	2	1
digit range	[0:3]	[0:2]	[0:1]	[0:0]
digit position (msd to lsd)	0	1	2	3
digit position (lsd to msd)	3	2	1	0
multiplier	$6 = 3!$	$2 = 2!$	$1 = 1!$	$1 = 0!$

In factorial base, the radix, that is, the upper bound of the range for possible digits, is determined by the position from least to most significative digit. The multipliers also change with this position.

Note that every number in the factorial number system ends with a 0. For this reason, this trailing 0 is sometimes omitted in its representation.

Permutations



Victor Vasarely

To understand the relationship between numbers in the factorial system and numbers in the decimal system, let us look first at the function

$$\phi : \text{factorial} \rightarrow \text{decimal}$$

that converts a number in factorial base to its decimal base.

This function is defined as follows:

$$\varphi(p) = \sum_{i=0}^{n-1} p_i \cdot i!$$

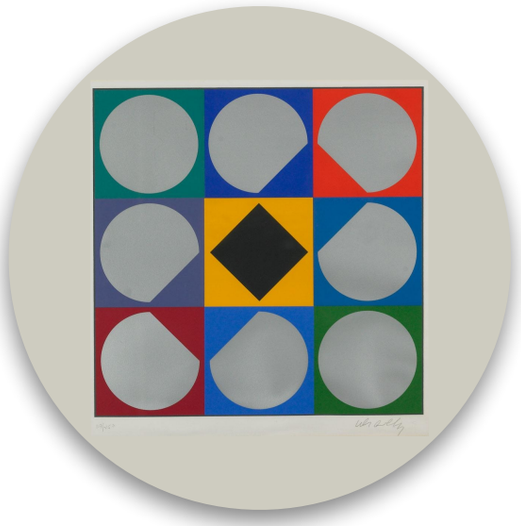
where $p_i \in [0:i]$ is the ith digit of p from least to most significative. The digit at position i has radix $i+1$, so it ranges over $[0:i+1)$. This matches the digit-range row of the table on the previous slide.

In brief, to transform a factorial number into a decimal number, we multiply its digits by their corresponding factorials. For example,

$$\varphi(3210) = 3 \cdot 3! + 2 \cdot 2! + 1 \cdot 1! + 0 \cdot 0! = 23$$

The situation here is similar to the case of binary or decimal numbers.

Permutations



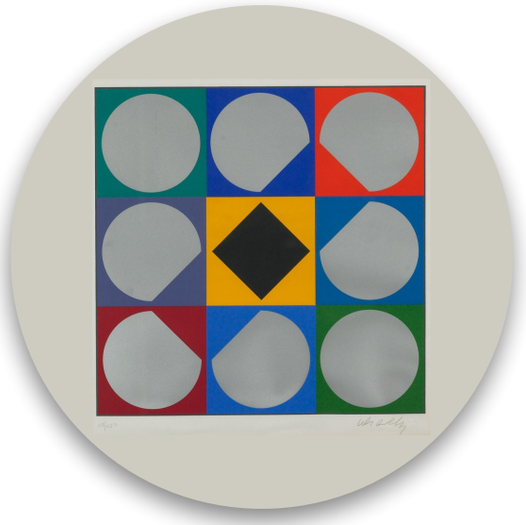
Victor Vasarely

```
def fac2dec(f: int) -> int:
    """
    the decimal value of a factorial number
    """
    # require: f is a factorial number


    # to be completed in class


    return d
    # ensure: f is the factorial representation of d
```

PERMUTATIONS



Victor Vasarely

To transform a decimal number to its factorial base, we note the following.

Every decimal number $d \in [0 : n!)$ can be written as:

$$d = \sum_{i=0}^{n-1} (p_i \cdot i!) = p_{n-1} \cdot (n-1)! + \ldots + p_1 \cdot 1! + p_0 \cdot 0!$$

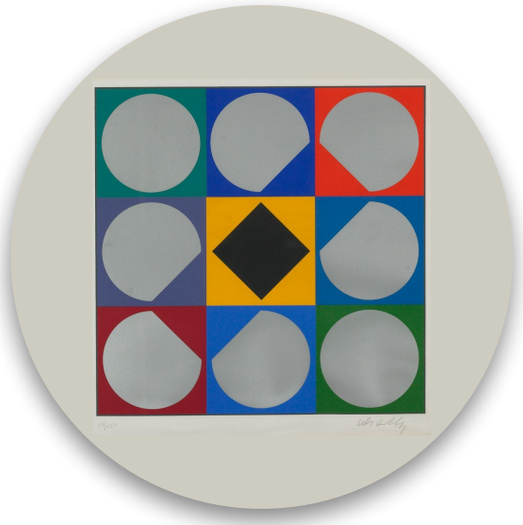
where each $p_i \in [0 : i]$ is the ith digit, from least to most significative, of a factorial number p. We call p the factorial representation of d.

The digits $p_0, p_1, \ldots, p_{n-1}$ are unique: there is exactly one way to write any d in this form. The reason is a counting argument: the radix of the digits of a factorial number with n digits imply we can write at most

$$\prod_{i=0}^{n-1} (i+1) = 1 \cdot 2 \cdot 3 \cdots n = n!$$

different factorial numbers, which equals exactly the size of $[0 : n!)$, so the representations cover every value in the range exactly once.

Permutations



Victor Vasarely

To transform a decimal number to its factorial base, we define a function

$$\delta\colon \text{decimal} \to \text{factorial}$$

which retrieves the digits p_i of the factorial representation p of a decimal number d.

The key observation is that each digit p_i can be read off directly:

$$p_i = (d \div i!) \bmod (i+1)$$

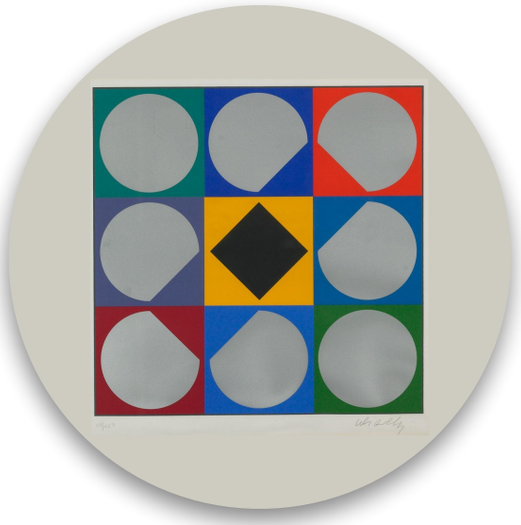
Moreover, rather than recomputing $d \div i!$ from scratch at each step, we compute p_i incrementally using a successive-division algorithm.

Define the running quotient $q_i = d \div i!$, starting from $q_0 = d$. Then at each step:

$$p_i \leftarrow q_i \bmod (i+1) \qquad \text{and} \qquad q_{i+1} \leftarrow q_i \div (i+1)$$

This works because dividing the running quotient by $i+1$ extracts both p_i as the remainder and advances the quotient to q_{i+1} in a single operation, so no factorial needs to be computed explicitly. Repeating for $i = 0,1,2,\ldots$ extracts all digits from least significant to most significant until the quotient reaches 0.

PERMUTATIONS



Victor Vasarely

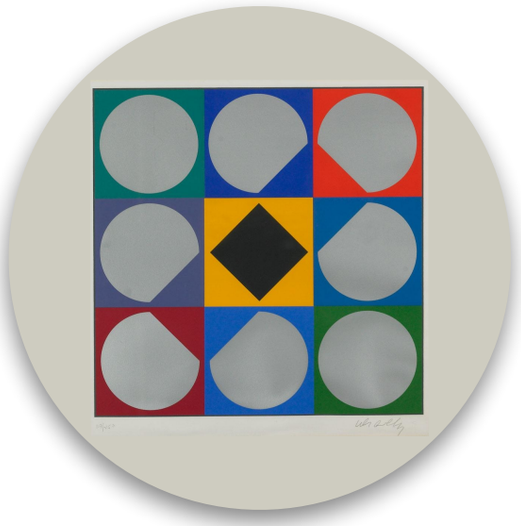
Let us calculate $\delta(23) = 3210$

i	$i!$	$r \div i!$	q_i	$q_i \bmod (i+1)$	p_i
0	1	$23 \div 1 = 23$	23	$23 \bmod 1 = 0$	0
1	1	$23 \div 1 = 23$	23	$23 \bmod 2 = 1$	1
2	2	$23 \div 2 = 11$	11	$11 \bmod 3 = 2$	2
3	6	$23 \div 6 = 3$	3	$3 \bmod 6 = 3$	3

Note that since $0! = 1! = 1$, rows $i \in \{0,1\}$ both show $r \div i! = 23$.

This table shows that the factorial representation of the decimal number 23 is 3210, where the digits p_i are read from most to least significant; namely, $p_3 p_2 p_1 p_0 = 3210$.

Permutations



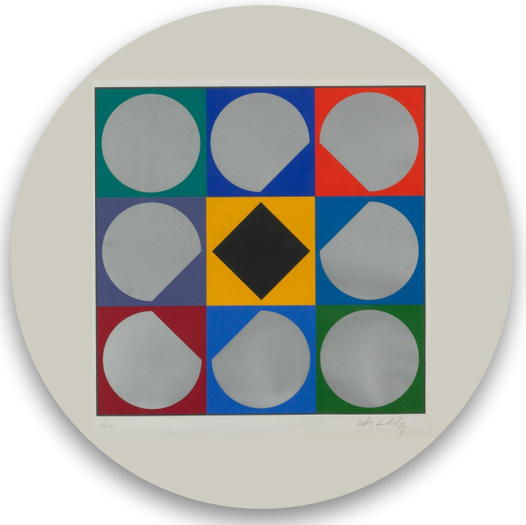
Victor Vasarely

```
def dec2fac(d: int) -> int:
    """
    the factorial representation of a decimal number
    """
    # require: d is a decimal number


    # to be completed in class


    return f
    # require: f is the factorial representation of d
```

Permutations



Victor Vasarely

The factorial number system is our first step toward understanding ranking and unranking algorithms for permutations. The second step involves Lehmer codes, named in reference to D. H. Lehmer.

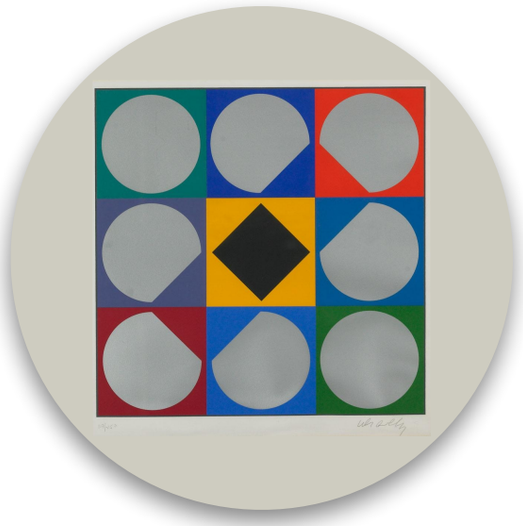
A Lehmer code gives us an alternative view a permutation. Instead of taking a permutation as a sequence of distinct elements, we encode it as a sequence whose elements at position i record how many elements (strictly) smaller than π_i appear to its right.

Formally, the Lehmer encoding of a permutation $\pi \in \Pi^n$ is a sequence $\rho \in [0:n)^n$ such that: for all $i \in [0:n)$:

$$\rho_i = \text{how many elements smaller than } \pi_i \text{ are in } \pi(i:)$$

where $\pi(i:) = [\pi_{i+1}, \pi_{i+2}, \ldots, \pi_{n-1}]$ denotes the suffix of π starting after position i.

Permutations



Victor Vasarely

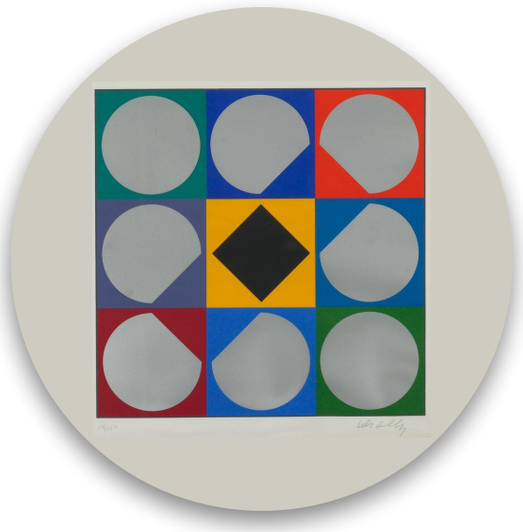
For example, consider the permutation $\pi = [2,0,3,1] \in \Pi^4$

i	π_i	$\pi(i{:}]$	elements in $\pi(i{:}]$ smaller than π_i	ρ_i
0	2	[0,3,1]	{0, 1}	2
1	0	[3,1]	{}	0
2	3	[1]	{1}	1
3	1	[]	{}	0

Therefore, the Lehmer code for $\pi = [2,0,3,1] \in \Pi^4$ is $\rho = [2,0,1,0]$.

For a larger example, the Lehmer code for $\pi = [1,5,0,6,3,4,2] \in \Pi^7$, is $\rho = [1,4,0,3,1,1,0]$. (Exercise: verify each entry.)

Permutations



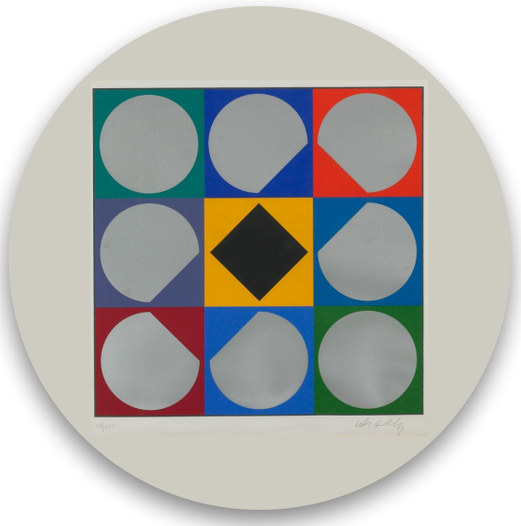
Victor Vasarely

To compute the Lehmer code of a permutation $\pi \in \Pi^n$ we:

1. scan π from left to right
2. at each step i compute the suffix $\pi(i:]$
3. set ρ_i to the number of elements in the suffix strictly smaller than π_i

In general, this algorithm will have complexity: $O(n^2)$, as there is a need to have two nested loops each of length at most n.

Subsets



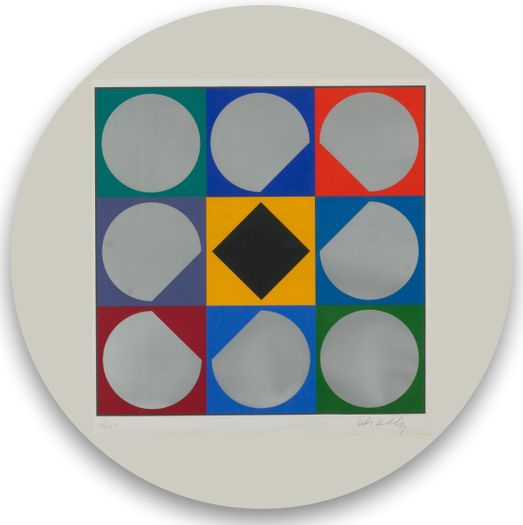
Victor Vasarely

```
def encode(pi:list[int], n: int) -> list[int]:
    """
    the Lehmer code of pi
    """
    # require: pi in Pi^n


    # to be completed in class


    return rho
    # ensure: rho is the Lehmer code of pi
```

Permutations



Victor Vasarely

To recover a permutation from a Lehmer code, we reason as follows.

Given a Lehmer code $\rho \in [0:n)^n$ with $\rho_i \in [0:n-i)$, we:

1. build a list $\alpha = [0,1,\ldots,n-1]$
2. scan ρ from left to right
3. at each step i, we set π_i to $\alpha[\rho_i]$ and remove ρ_i from α

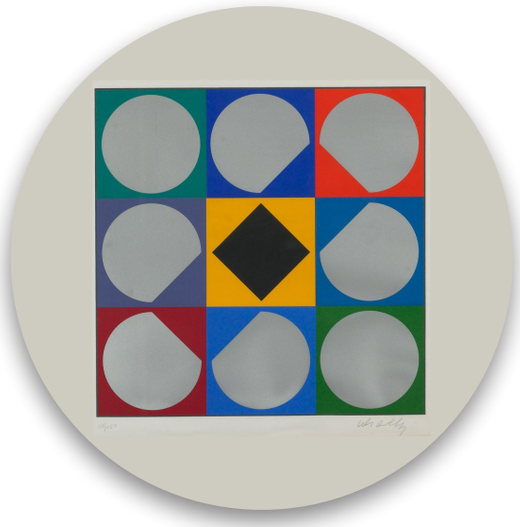
For example, let $\rho = [2,0,1,0]$. Then,

i	ρ_i	α	$\pi_i = \alpha[\rho_i]$
0	2	[0,1,2,3]	2
1	0	[0,1,3]	0
2	1	[1,3]	3
3	0	[1]	1

Then, $\pi = [2,0,3,1]$, and we recovered the original permutation exactly.

This algorithm has $O(n^2)$ complexity as each removal from a list costs $O(n)$ with a plain list.

PERMUTATIONS



Victor Vasarely

To see why the decoding algorithm works, we note the following.

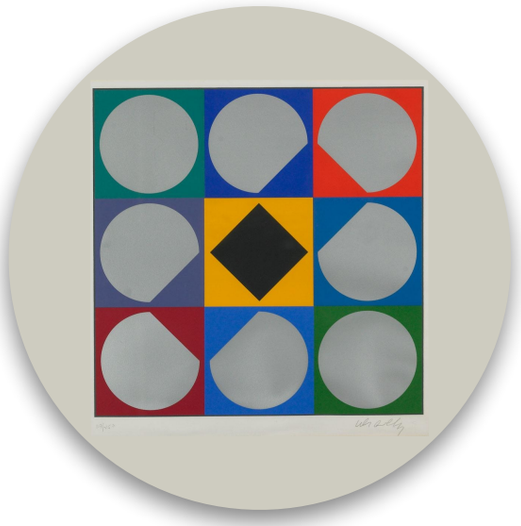
The correctness rests on a single invariant:

> the elements still in α at step i are exactly the values that will occupy positions $i, i+1, \ldots, n-1$ in the permutation π.

The invariant holds initially. At step $i = 0$, we have $\pi = []$ and $\alpha = [0,1,\ldots,n-1]$. Cleary, α contains all values that will appear at positions $0,\ldots,n-1$ of π.

The invariant is preserved at each step. Assuming the invariant holds at step i, all elements smaller than π_i that have not yet been placed must appear at positions $i+1,\ldots,n-1$ are still in α. This implies $|\alpha| \leq \pi_i$, which, by definition of a Lehmer code ρ_i. This means π_i is the $(\rho_i+1)^{\text{th}}$ smallest element in α, namely, $\pi_i = \alpha[\rho_i]$. Removing π_i from α restores the invariant for step $i+1$.

Subsets



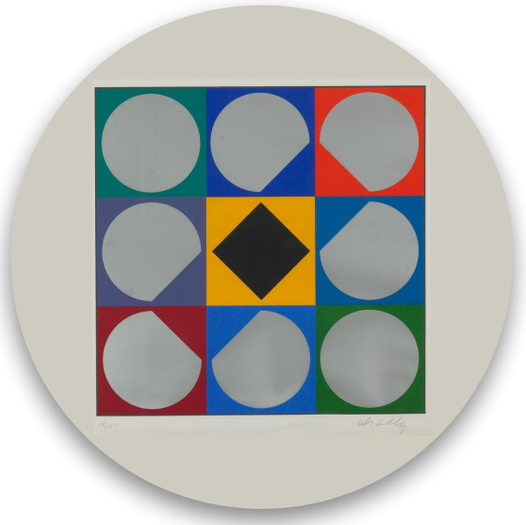
Victor Vasarely

```
def decode(rho: list[int], n: int) -> list[int]:
    """
    the permutation in position r in lexorder
    """
    # require: rho a Lehmer code


    # to be completed in class


    return pi
    # ensure: rho is the Lehmer code of pi
```

Permutations

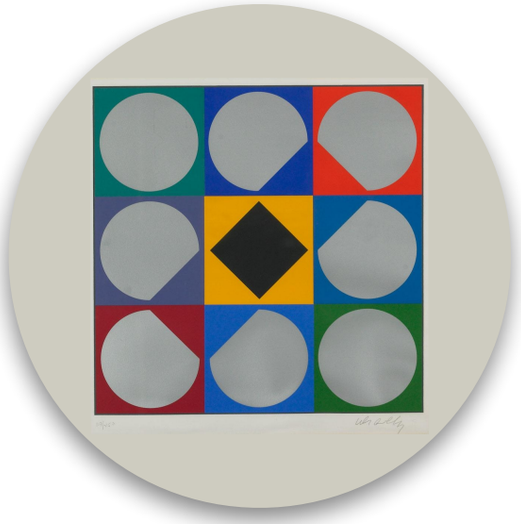


Victor Vasarely

It can easily be checked that encoding then decoding returns the original permutation, and decoding then encoding does too. In other words, **encode** and **decode** are inverses of each other.

This yields a bijection between Lehmer codes and permutations.

Permutations



Victor Vasarely

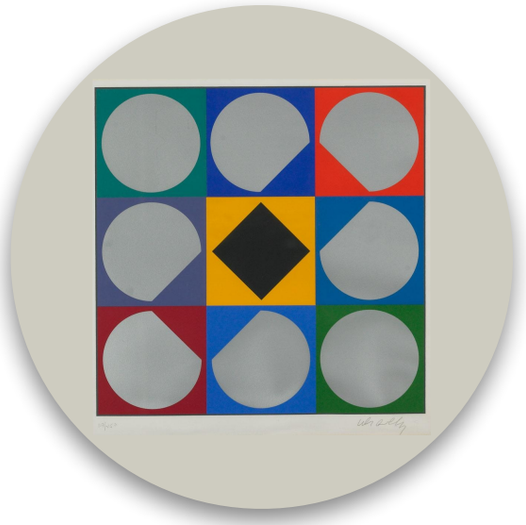
The Lehmer encoding of a permutation ties back to factorial numbers.

Note that ρ_i counts how many elements smaller than π_i appear to its right. Since at position i of π we may have, at most, $n-1-i$ strictly smaller elements still to the right, ρ_i can take any value in $[0:n-i)$. This is exactly the digit range of position i in the factorial number system.

position i	factoradic digit range	elements to the right	factoradic radix	digit range of ρ_i
0 (MSB)	n-1	$[0:n)$	n	$[0:n)$
1	n-2	$[0:n)$	n-1	$[0:n)$
⋮	⋮	⋮	⋮	⋮
n-2	1	$[0:2)$	2	$[0:2)$
n-1 (LSB)	0	$[0:1)$	1	$[0:1)$

The digit ranges match at every position. In addition, there are $n!$ permutations and $n!$ factorial numbers with n digits. Thus, the correspondence is a bijection: Lehmer codes are factorial numbers.

Permutations



Victor Vasarely

This bijection between Lehmer codes and factorial numbers is the bridge between permutations and natural numbers:

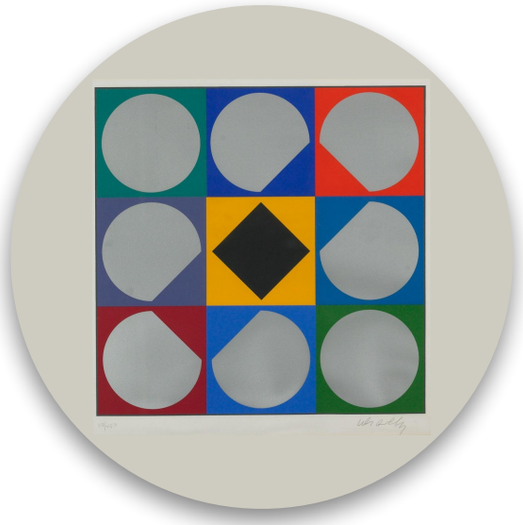
$$[0:n!) \underset{\varphi}{\overset{\delta}{\longleftrightarrow}} \text{Lehmer codes} \underset{\text{decode}}{\overset{\text{encode}}{\longleftrightarrow}} \Pi^n$$

The upper arrow on each step goes left to right; the lower arrow goes right to left. The four named operations are:

- δ converts a decimal number to its factorial representation (`dec2fac`)
- φ converts a factorial number to its decimal value (`fac2dec`)
- decode converts a Lehmer code to a permutation
- encode converts a permutation to its Lehmer code

Each operation is its own algorithm. The arrow diagram shows how they compose to give rank and unrank.

Permutations



Victor Vasarely

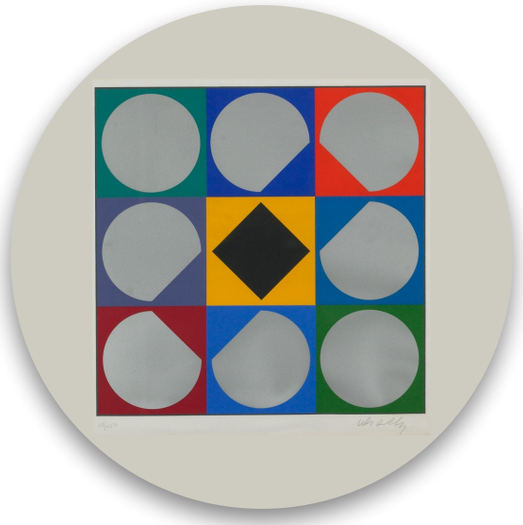
So far we have presented δ/φ and $\mathbf{encode/decode}$ as bijections. They are, however, something stronger: they are order-preserving bijections.

Concretely, let $\prec$ be the symbol for lexicographic order:

- δ/φ preserve the natural order on $[0:n!)$ and the lexicographic order on Lehmer codes: $r < r'$ if and only if $\delta(r) \prec \delta(r')$.

- $\mathbf{encode/decode}$ preserves the lexicographic order on Π^n and the lexicographic order on Lehmer codes: $\pi \prec \pi'$ if and only if $\mathbf{encode}(\pi) \prec \mathbf{encode}(\pi')$.

Permutations



Victor Vasarely

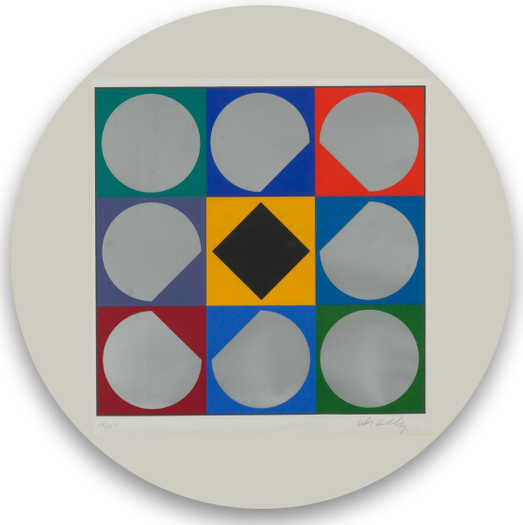
To see why δ/φ preserve order, note that:

> The factorial number system is a positional system with the MSD at position 0. Comparing two factorial numbers p and p' lexicographically gives the same result as comparing their decimal values $\varphi(p)$ and $\varphi(p')$, because higher positions carry larger weight. This is the same argument as for decimal numbers.

To see why $\mathrm{encode}/\mathrm{decode}$ preserve order, note that:

> Let $\pi, \pi' \in \Pi^n$ with $\pi \prec \pi'$, and let i be the first position where they differ, so $\pi_i < \pi'_i$. Since $\pi_j = \pi'_j$ for all $j \in [0:i)$, exactly the same elements have been placed in positions $[0:i)$ in both permutations. This implies, the same elements remain available at position i, and the count of available elements smaller than π_i is less than the count smaller than π'_i. In other words, $\rho_j = \rho'_j$ for all $j \in [0:i)$, and $\rho_i < \rho'_i$. Therefore, $\rho \prec \rho'$. The argument reverses for decode.

Permutations



Victor Vasarely

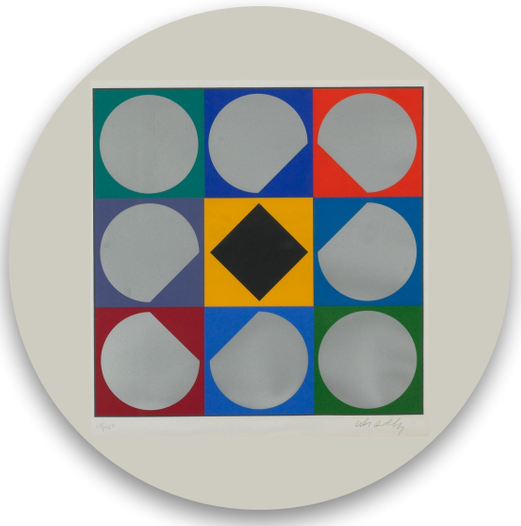
Then, we have:

1. δ/φ are order preserving bijections; and
2. $\mathrm{encode/decode}$ are also order preserving bijections.

Together, these two facts imply:

- $\mathrm{rank} = \mathrm{encode}; \varphi$ maps Π^n in lexicographic order exactly onto $[0:n!)$ in numerical order.
- $\mathrm{unrank} = \delta; \mathrm{decode}$ maps $[0:n!)$ in numerical order exactly onto Π^n in lexicographic order

Subsets



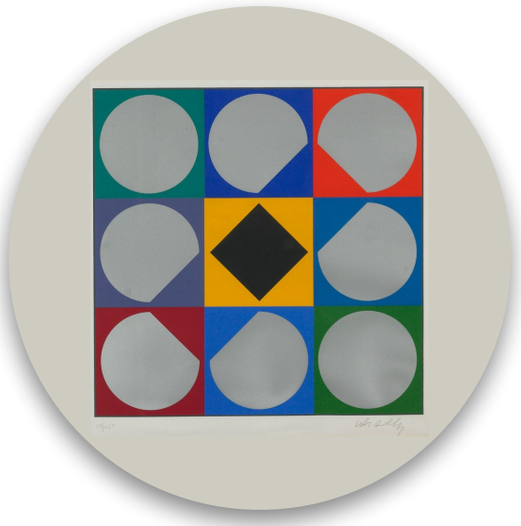
Victor Vasarely

```
def rank(pi:list[int], n: int) -> int:
    """
    the position of a permutation in lexorder
    """
    # require: pi in Pi^n


    # to be completed in class


    return r
    # ensure: Pi^n_r = pi
```

Subsets



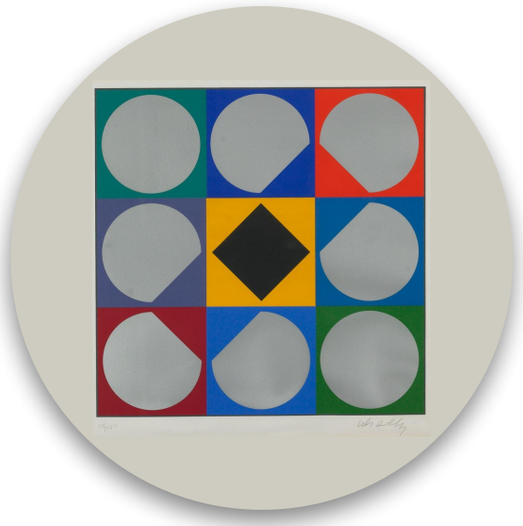
Victor Vasarely

```
def unrank(r: int, n: int) -> list[int]:
    """
    the permutation in position r in lexorder
    """
    # require: r in [0:n!)


    # to be completed in class


    return pi
    # ensure: Pi^n_r = pi
```

Permutations



Victor Vasarely

We can also think of the rank function as being defined recursively as:

$$\text{rank}([\pi_0],1) = 0$$

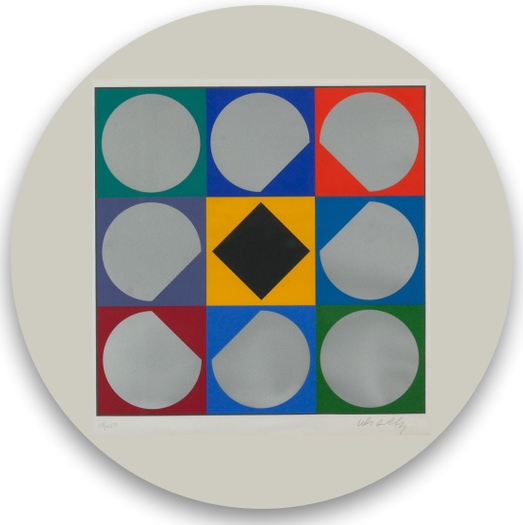
$$\text{rank}(\pi, n) = \pi_0 \cdot (n-1)! + \text{rank}(\pi', n-1)$$

group

offset

where, for $i \in [0 : n-1)$, we have $\pi'_i = \pi_{i+1} - (1 \text{ if } \pi_{i+1} > \pi_0 \text{ else } 0)$.

PERMUTATIONS



Victor Vasarely

The correctness from the recursive definition of rank stems from the following facts.

Recall that rank reads the bijection diagram right to left: encode π to its Lehmer code ρ, then evaluate $\varphi(\rho)$. Expanding:

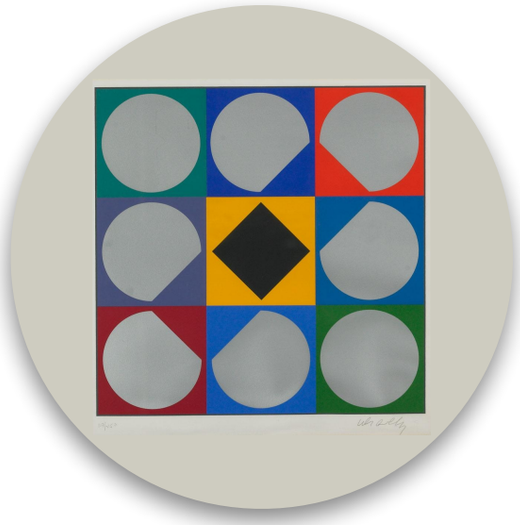
$$\text{rank}(\pi) = \varphi(\rho) = \sum_{i=0}^{n-1} \rho_i \cdot (n-1-i)!$$

Fact 1. The "group" corresponds to the largest term in the sum, that is, the first term, that is, $\rho_0 \cdot (n-1)!$. We want to find out ρ_0 in terms of the elements in π. We reason as follows:

- ρ_0 = the number of elements smaller than π_0 to its right,
- = the number of elements in $[0:\pi_0)$ still available after position 0
- = π_0 itself (since all of $[0:n)$ is available at the start).

This implies $\rho_0 = \pi_0$.

PERMUTATIONS



Victor Vasarely

Fact 2. The "offset" corresponds to the rank of some permutation $\pi' \in \Pi^{n-1}$. We want to find out the elements in π' in terms of the elements in π . We reason as follows.

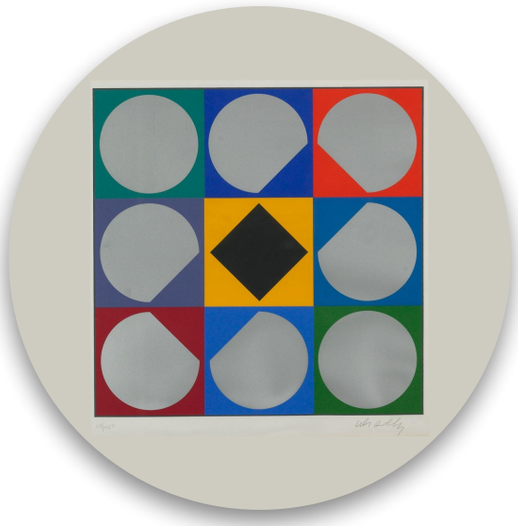
We know that the "offset" corresponds to the remaining terms

$$\sum_{i=1}^{n-1} \rho_i \cdot (n-1-i)!$$

and that this value corresponds to the rank of a permutation $\pi' \in \Pi^{n-1}$, obtained by removing π_0 from π and decrementing every element larger than π_0 by 1.

The reason the decrement is exactly 1 is that removing π_0 creates a single gap in the value set: the remaining values are a subset of $[0:n)$ missing only π_0. The values below π_0, the set $L = [0:\pi_0)$, are already consecutive from 0 and need no adjustment. The values above π_0, the set $U = (\pi_0{+}1 : n{-}1)$, needs to map to $[\pi_0 : n{-}1)$. Each such value sits exactly 1 above its target value. Subtracting 1 from each closes the gap exactly and produces the correct permutation $\pi' \in \Pi^{n-1}$.

Subsets



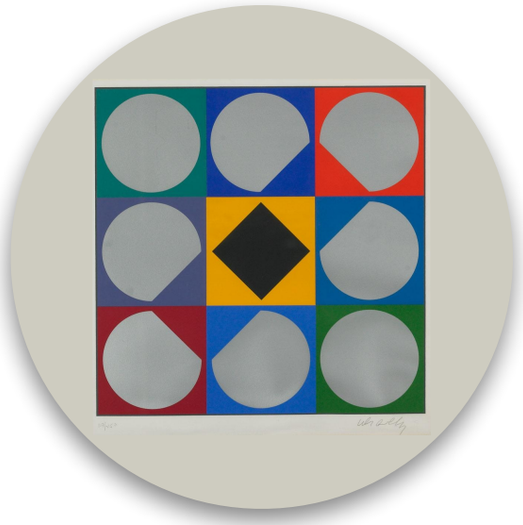
Victor Vasarely

```
def rrank(pi:list[int], n: int) -> int:
    """
    the position of a permutation in lexorder
    """
    # require: pi in Pi^n


    # to be completed in class


    return r
    # ensure: Pi^n_r = pi
```

PERMUTATIONS



Victor Vasarely

The recursive implementation of unrank can be understood as follows.

Unrank reads the bijection diagram left to right: apply $\delta(r)$ to get the Lehmer code ρ, then decode ρ to get the permutation π. The recursion comes from expanding decode.

At each step the first element of π is determined by $\rho_0 = r \div (n-1)!$. Precisely, ρ_0 is the index of the current available list $\alpha = [0,\ldots,n-1]$, and the remainder $r' = r \bmod (n-1)!$ is the rank of the tail.

Then:

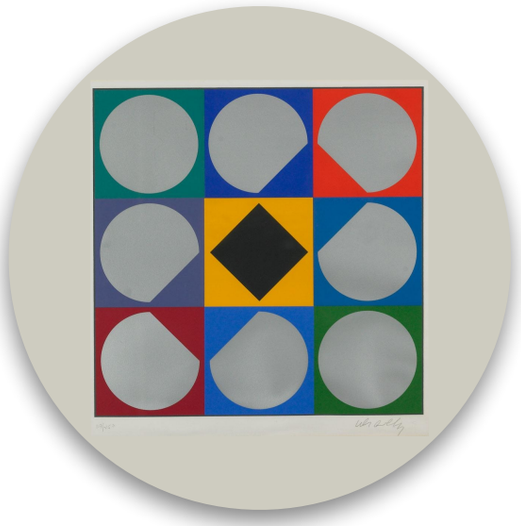
$$\begin{aligned} \text{unrank}(r, [\,], 0) &= [\,] \\ \text{unrank}(r, \alpha, k) &= [\alpha[q]\ ] + \text{unrank}(r', \text{del}(\alpha, \alpha[q]), k-1) \end{aligned}$$

where $\quad q = r \div (k-1)!$

$\qquad\quad r' = r \bmod (k-1)!$

Intuitively, at each level of the recursion: divide r by $(k-1)!$ to obtain the index j of α; the next element of π is $\alpha[j]$; recurse on the remainder $r \bmod (k-1)!$ with $\alpha[j]$ removed from α and k decremented by 1.

Subsets



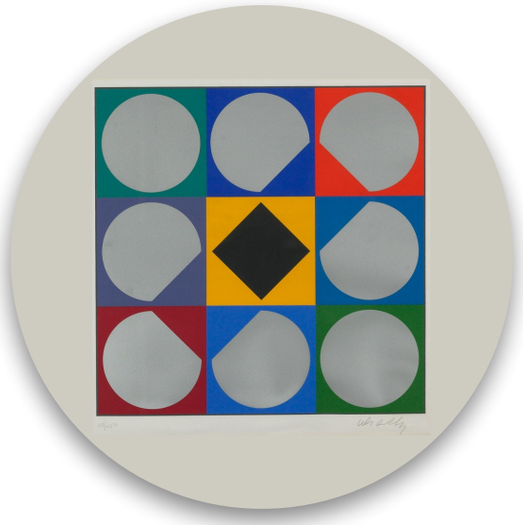
Victor Vasarely

```
def uunrank(r: int, n: int) -> list[int]:
    """
    the permutation in position r in lexorder
    """
    # require: r in [0:n!)


    # to be completed in class


    return pi
    # ensure: Pi^n_r = pi
```

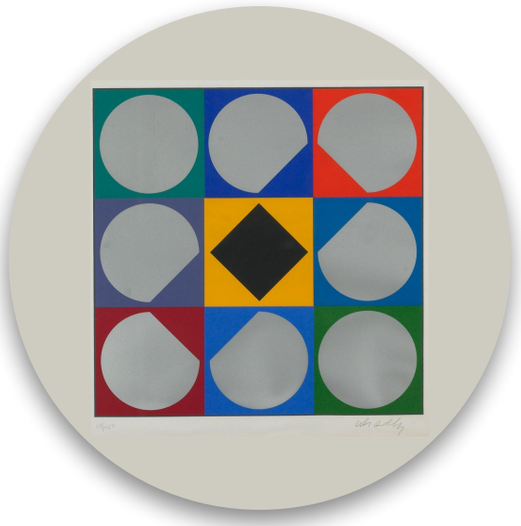
Permutations



Victor Vasarely

- The Lehmer code ρ of a permutation π records at each position i how many elements smaller than π_i appear in $\pi(i:]$. This implies $\rho_i \in [0:n-i)$
- The digit ranges $[0:n-i)$ match the factorial number system exactly: a Lehmer code is a factorial number, and the correspondence is a bijection
- This bijection decomposes rank and unrank into two steps each:
  - rank: encode π to ρ, then $\varphi(\rho)$ gives r;
  - unrank: $\delta(r)$ gives ρ, then decode ρ to π
- The encode and decode operations (Lehmer ↔ permutation) are the remaining pieces to implement.

Permutations



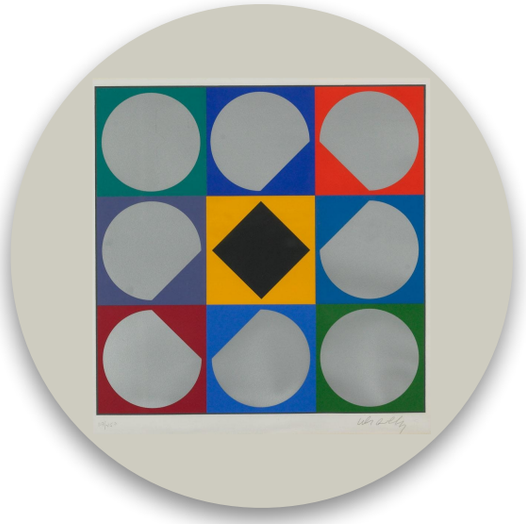
Victor Vasarely

```
def successor(pi:list[int], n:int) -> list[int]:
    """
    the next permutation in lexorder
    """
    # require: pi in Pi^n


    # to be completed in class


    return pi
    # ensure: rank(rho) = rank(pi)+1 mod n!
```

Permutations



Victor Vasarely

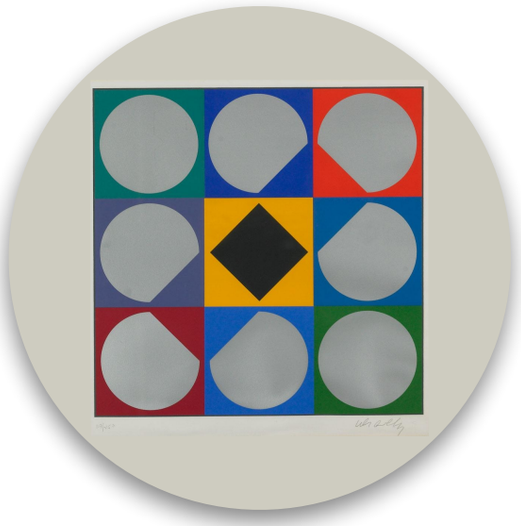
For the successor algorithm for permutations notice the following.

Let $\pi \in \Pi^n$. In addition, let i be the largest index of π for which $\pi_i < \pi_{(i+1)}$ and j be the largest index of π for which $\pi_i < \pi_j$.

It is clear from these assumptions that all the elements immediately after i form a strictly decreasing sequence. This implies that no permutation of these elements will make it advance in the order. The index j then tells us which of the elements in this sequence is the smallest value which will advance π in the order.

If we swap the values of π_i and π_j, then, the sequence $\pi(i:]$ is in decreasing order. Reversing this sequence produces the next lexicographical minimal permutation.

Permutations



Victor Vasarely

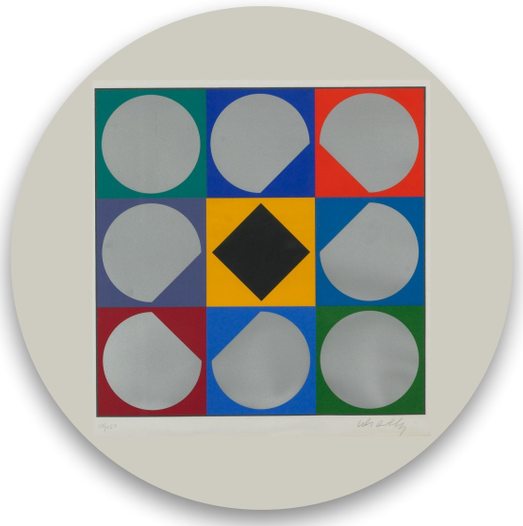
To understand the principle that guides the previous argument. Let a, b, c, d stand for $0,1,2,3$, respectively. Then Π^4 is the dictionary of all $4! = 24$ words that use each of the four letters exactly once.

Consider the following window of Π^4:

rank	π	word
8	[1,2,0,3]	*bcad*
9	[1,2,3,0]	*bcda*
10	[1,3,0,2]	*bdac*
11	[1,3,2,0]	*bdca*
12	[2,0,1,3]	*cabd*
13	[2,0,3,1]	*cadb*

Question: What word comes right after *bdca*? How much of the word is kept from one entry to the next? What happens when we get from rank 11 to rank 12?

Permutations



Victor Vasarely

Three greedy choices. To find the next word in the dictionary we can:

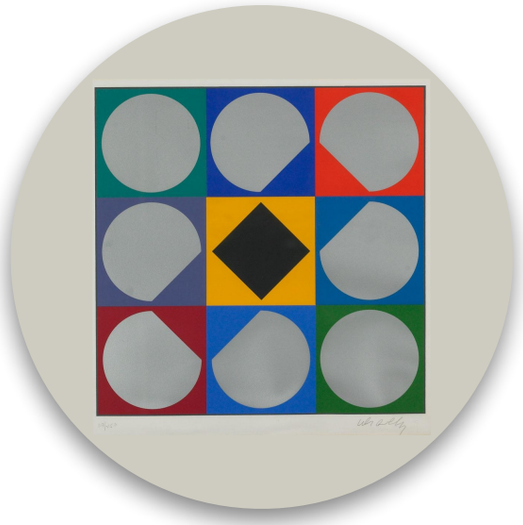
1. Keep the longest possible prefix. Change **as far to the right as possible**.
2. Raise the first changed position **by the least possible amount**.
3. Fill in **the rest as small as possible**.

Why the longest prefix? Lexorder compares position by position, from the left. This means all words that share a given prefix sit together in one contiguous block: every word starting with bd ($bdac$,$bdca$) is on one page, inside the larger block of words starting with b.

Suppose a larger word τ than π exists that shares a long prefix with π, and a larger word τ' that shares only a shorter one. Then τ' differs from π at an earlier position, and has a larger letter there (otherwise it would not be larger than π). But τ agrees with π at that position, so $\tau < \tau'$. Leaving the block means jumping past everything inside it.

So the next word must share as long a prefix with π as any larger word can.

PERMUTATIONS



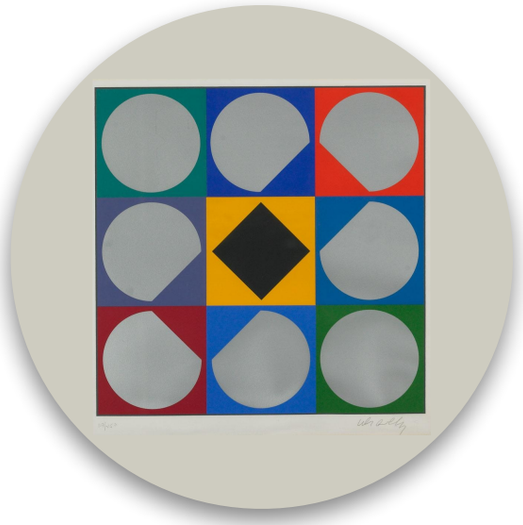
Victor Vasarely

THREE GREEDY CHOICES. Let us try this approach on $bdca = [1,3,2,0]$:

- keep bdc: Since a is the only letter left, we have $bdca$ is the only word with that prefix. There is nothing larger. We drop the prefix.
- keep bd: The words starting with bd are $bdac$ and $bdca$. We have $bdca$ is the later one, because ca is the largest arrangement of $\{a, c\}$. There is nothing larger. We drop the prefix.
- keep b: The words starting with b run $bacd$, $badc$, $bcad$, $bcda$, $bdac$, $bdca$. Ours is the last one, because dca is the largest arrangement of $\{a, c, d\}$. There is nothing larger. We drop the prefix.
- keep $[]$: The first letter (that is, b) can still rise. We choose the next larger available letter, c. We fill $\{a, b, d\}$ in the smallest possible way: abd. The result then becomes: $cabd$.

The **key observation** is: a block written in decreasing order is the largest arrangement of its elements, so it cannot be advanced.

PERMUTATIONS

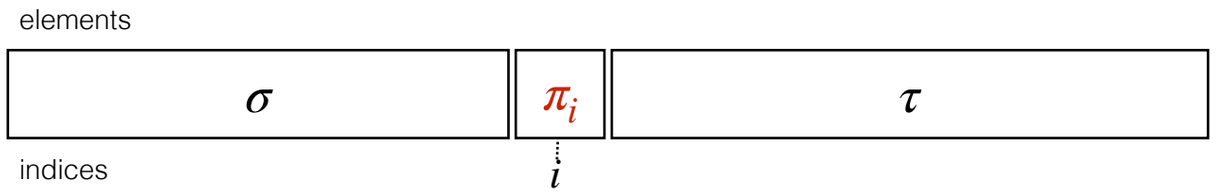


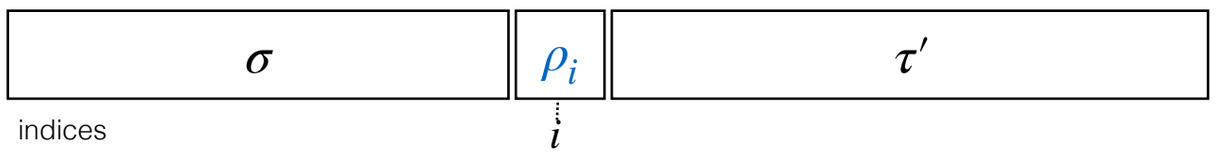
Victor Vasarely

The idea is the following. Consider permutations π and ρ such that

$$\text{rank}(\rho) = (\text{rank}(\pi) + 1) \bmod n!$$

We know that:

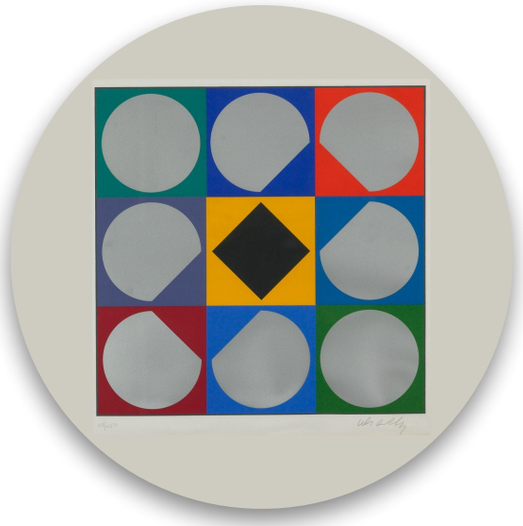
$\pi =$ 

$\rho =$ 

Let σ be the longest common prefix between π and ρ. Then:

1. $\pi_i \cup \{x \mid x \in \tau\} = \rho_i \cup \{y \mid y \in \tau'\}$
2. τ is sorted decreasingly
3. τ' is sorted increasingly.
4. $\rho_i = \min\{y \mid y \in \tau \text{ and } y > \pi_i\}$

Permutations



Victor Vasarely

How do we find the position i that must change? A strictly decreasing block cannot be advanced: rearranging only its elements never gives a larger permutation.

Why? A decreasing block is the largest arrangement of its elements. To make a block as large as possible, lexicographic order says to put the largest element first (the first position is compared first), then the largest of those left, and so on. That is exactly decreasing order.

To get a larger permutation, something to the left of the block must change.

Algorithmically, we can scan from the right while the permutation is decreasing, and stop at the first ascent. Then:

$$i = \text{largest index such that } \pi_i < \pi_{i+1}$$

Permutations



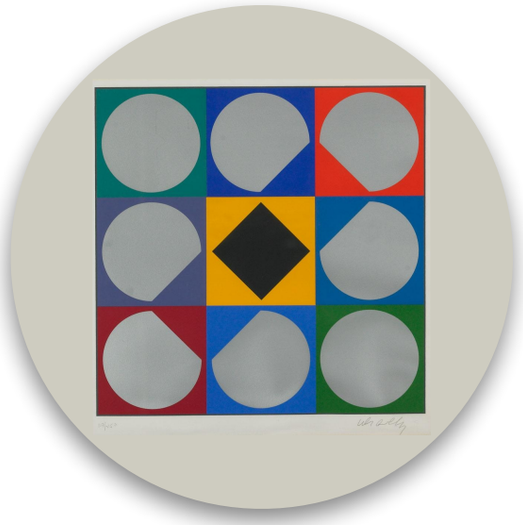
Victor Vasarely

The prefix is fixed by i, so the new value at position i must come from the suffix, and it must be the smallest suffix element larger than π_i.

The suffix is decreasing, so its values shrink from left to right. Hence the smallest value that is still larger than π_i is the rightmost such value. Then:

$$j = \text{the largest index with } \pi_j > \pi_i$$

Permutations



Victor Vasarely

Having found the indices i and j, we compute the successor ρ of π as follows:

1. Set $\rho = \pi$

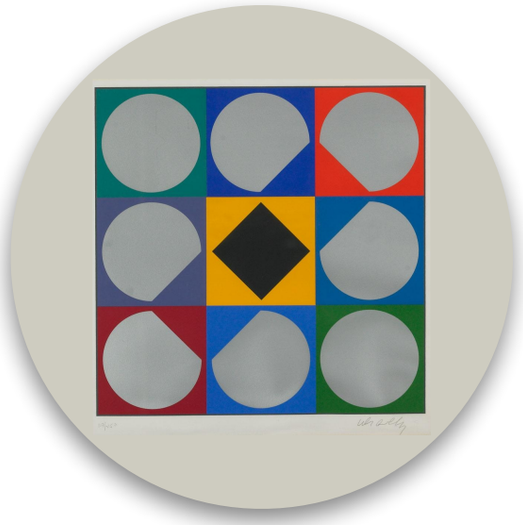
2. Swap ρ_i and ρ_j. If we interchange the values at positions i and j in ρ, we obtain that the new value ρ_i is the smallest possible larger value than π_i.

   The tail is still decreasing after the swap. We have $\rho_j = \pi_i$ and $\rho_i = \pi_j$. Since j was the largest index with a larger value:

   - $\rho_{j-1} > \rho_j > \rho_i$, that is, a larger value on its left, and
   - $\rho_{j+1} < \rho_i$, that is, a smaller value on its right.

3. We still need to ensure that the tail $\rho(i:]$ is as small as possible, that is, it is in increasing order. We do not need to sort. Reversing the tail turns a decreasing sequence into an increasing one in $O(n)$.

PERMUTATIONS



Victor Vasarely

For example. Let $\pi = [1,3,4,2,0]$. We have:

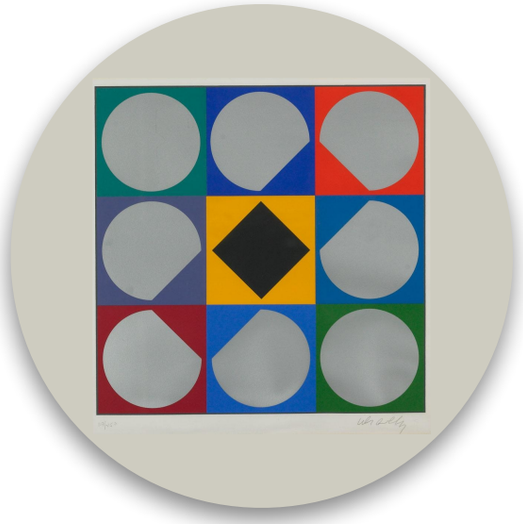
i	$\pi[i\text{-}1]$	$\pi[i]$	$\pi[i\text{-}1] > \pi[i]$
4	2	0	true
3	4	2	true
2	3	4	false
1	1	3	
0		1	

Then:

1. $i = 1$.
2. the prefix $\pi[0:1] = [1]$ stays;
3. $\pi_i = 3$ must change;
4. the suffix $\pi[2:] = [4,2,0]$ is strictly decreasing, so nothing done inside it can advance π.

If no ascent, $\pi = [n-1,\ldots,0]$ is the last permutation (wrap-around).

PERMUTATIONS



Victor Vasarely

(cont.) We have $\pi = [1,3,4,2,0]$, $i = 1$, $\pi_i = 3$, and $\pi(i:) = [4,2,0]$.

j is not always next to i.

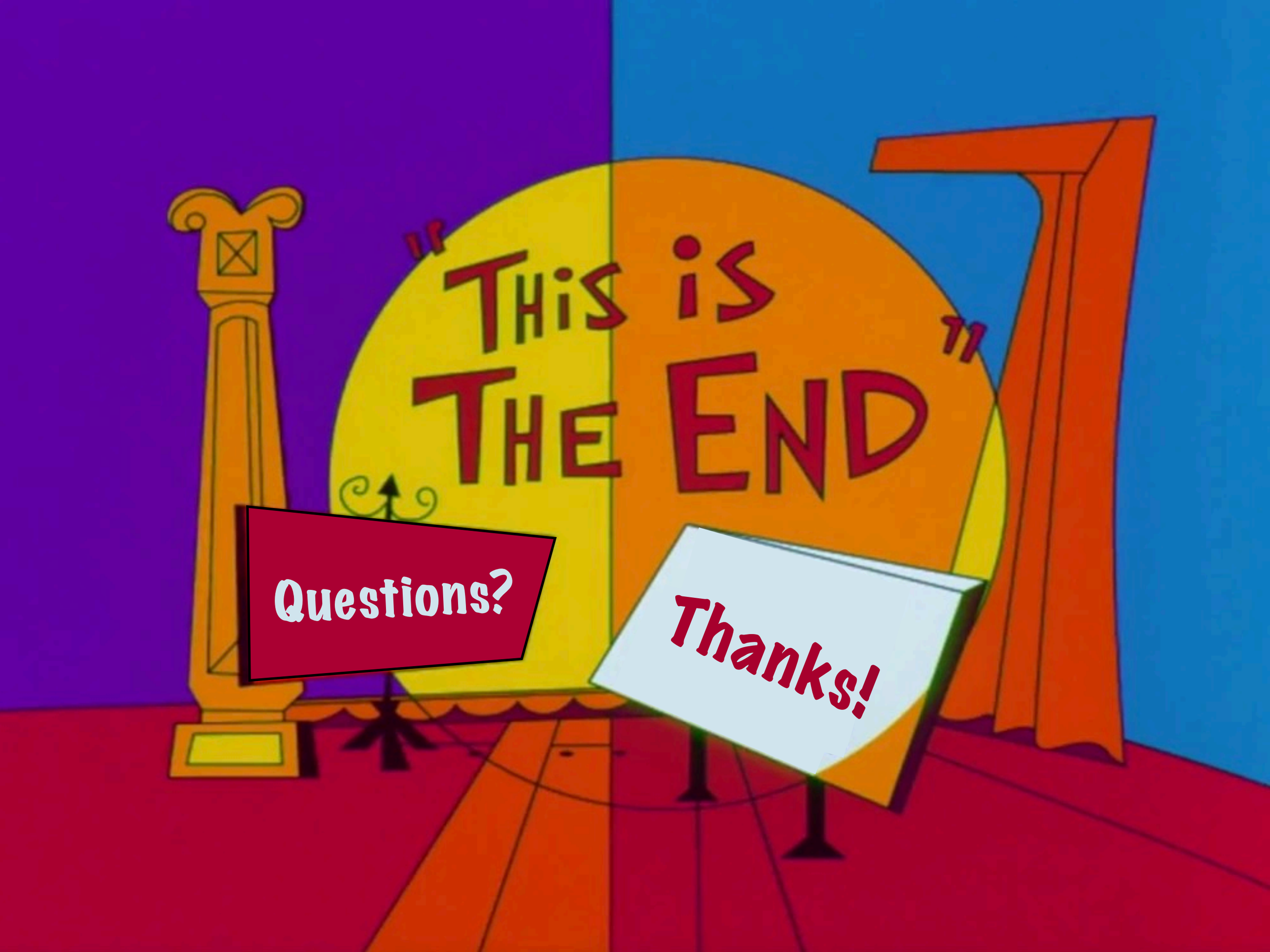
j	$\pi[j\text{-}1]$	$\pi[j]$	$\pi[j\text{-}1] > \pi[j]$
4	2	0	true
3	4	2	true
2	3	4	false
1	1	3	
0		1	

Now, j must be the largest index with $\pi_j > \pi_i = 3$. From this, $j = 2$.

We compute the successor ρ as follows:

i	j	ρ	
1	2	[1, 3, 4, 2, 0]	
		[1, 4, 3, 2, 0]	swap
		[1, 4, 0, 2, 3]	reverse

This means that the successor of $\pi = [1,3,4,2,0]$ is $\rho = [1,4,0,2,3]$.


"This is The End"
Questions?
Thanks!